Diagnosing JEPA World Models with Action-Conditioned Predictive Consistency

Anonymous Authors

July 2026

Abstract

Joint-embedding predictive architectures (JEPAs) learn world models that predict in a compact latent space rather than in pixels, reducing the pressure to model nuisance appearance. Yet this provides no guarantee against visual perturbations: they can still alter the encoded representation and affect subsequent action-conditioned predictions. Bisimulation captures this requirement precisely: two observations should be treated as the same state only when their action-conditioned consequences agree. Guided by this criterion, we introduce Action-Conditioned Predictive Consistency (ACPC), a diagnostic that measures how far a clean history and a visually perturbed view of it diverge after being rolled forward under the same action sequence. We prove that this divergence bounds the perturbation-induced change in multi-step prediction error and planner cost. Two complementary measures complete the diagnostic: the Invariance Radius quantifies how far a clean view and its perturbed counterpart spread after prediction, and the Separation Rate checks that different states remain distinguishable. Experiments on four visual control tasks and two model families show that pairwise ACPC predicts perturbation-induced prediction and cost changes, and that the ACPC-based screen transfers to new tasks and visual shifts.

Keywords: visual world models; predictive consistency; action-conditioned rollouts; visual invariance; checkpoint diagnostics.

Anonymous reproducibility note: code and data availability follow the venue's double-blind supplemental policy.

1 Introduction

World models support control by predicting the outcomes of candidate actions [1, 2]. Joint-embedding predictive architectures (JEPAs) predict future representations rather than reconstruct future pixels, avoiding an explicit requirement to reproduce nuisance visual details [3, 4, 5, 6, 7]. Such representations should be insensitive to task-irrelevant visual perturbations while preserving distinctions between situations that require different actions. This requirement echoes the intuition behind bisimulation, which defines state equivalence through action-conditioned consequences rather than visual appearance [8, 9]. However, encoder distances alone do not show how a perturbation propagates through prediction. Over a multi-step rollout, the predictor may amplify or contract the initial representation difference. Encoder-level and single-transition diagnostics do not directly measure this rollout-level effect. We therefore ask: when a clean history and its perturbed view are rolled forward under the same action sequence, how far apart are their predicted trajectories?

To measure this rollout-level effect, we introduce Action-Conditioned Predictive Consistency (ACPC), as illustrated in Figure 1. ACPC rolls a clean history and its perturbed view forward under the same action sequence and measures the distance between their predicted trajectories. Low ACPC alone, however, does not rule out representational collapse: a constant representation would make every paired rollout identical. We therefore summarize ACPC across histories with the *Invariance Radius* (IR), where lower values indicate less sensitivity to visual perturbations. The *Separation Rate* (SR) checks whether different states remain distinguishable after rollout, with higher values indicating better separation.

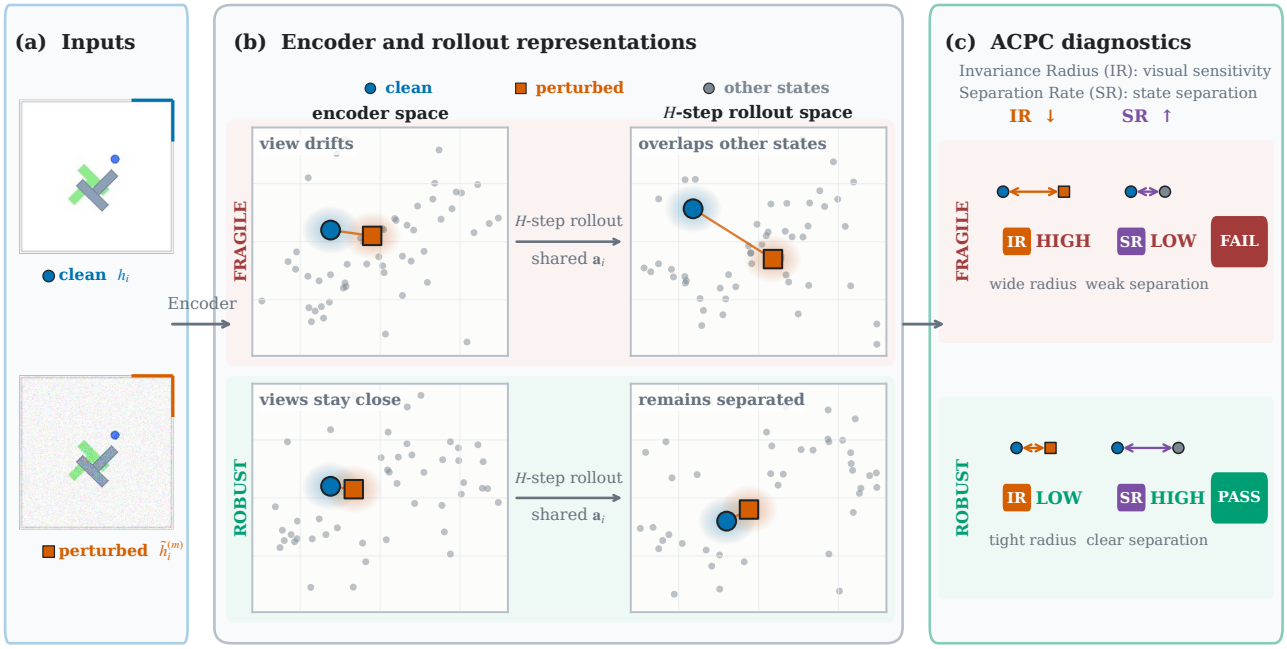

Figure 1: Overview of the Action-Conditioned Predictive Consistency (ACPC) diagnostic. (a) Inputs are logged PushT observations: a clean history and a Gaussian-perturbed view of the same history. **(b)** Schematic latent-space comparison of a fragile (top) and a robust (bottom) checkpoint. The frozen encoder embeds the paired inputs, and both branches are rolled out for H steps under anchor i's shared recorded action sequence $\mathbf{a}_i$; ACPC measures the distance between the clean and perturbed predicted trajectories. Under the fragile checkpoint, the perturbation-induced spread grows during the rollout and overlaps other states (gray); under the robust checkpoint, the paired views stay close while different states remain separated. Layouts are schematic: only within-panel proximity is meaningful, and positions enter no reported metric. **(c)** Checkpoint-level summary of each row. The Invariance Radius (IR) aggregates normalized ACPC over perturbation draws and anchors and is reported relative to the unaugmented checkpoint; the Separation Rate (SR) is the fraction of selected different-label pairs whose rollout distance exceeds the raw IR by a fixed margin. A checkpoint passes the screen only if relative IR is low and SR is high; passing is specific to the evaluated visual shift and state-coordinate labels and does not certify robustness.

We prove that ACPC bounds the perturbation-induced change in multi-step prediction error. When the same candidate action sequences are evaluated from both views, ACPC also yields bounds on changes in their predicted costs (Propositions 1 and 2). These bounds hold for each evaluated pair and require no distributional or smoothness assumptions. Experiments across four visual control tasks and two model families (LeWM [10] and PLDM [11, 12]) show that ACPC is informative about prediction-error change and selection regret for the cross-entropy method (CEM) [13]. Together, IR and SR help identify checkpoints that perform well under visual perturbations across tasks and perturbation types.

The contributions are:

1. We introduce **Action-Conditioned Predictive Consistency (ACPC)**, which compares the predicted trajectories of a clean history and its perturbed view under the same action sequence. IR summarizes sensitivity across histories, while SR checks whether different states remain distinguishable (Section 3).
2. We prove **samplewise bounds** on how much a visual perturbation can change multi-step prediction error. When the same candidate action sequences are evaluated from both views, we also bound changes in their predicted costs (Propositions 1 and 2).
3. We evaluate **ACPC, IR, and SR** across four control tasks, two model families, and three visual perturbations. The results show that ACPC provides predictive information about perturbation-induced changes in multi-step prediction error and planning cost, while IR and SR help identify trained models that maintain control performance (Section 4).

2 Related Work

World Models. World models predict how an environment evolves under actions and use those predictions for control. DreamerV3 [1] learns from imagined trajectories, while TD-MPC2 [2] plans with learned latent dynamics. Joint-embedding predictive architectures (JEPAs) learn representations by predicting targets in representation space rather than reconstructing pixels [3, 4, 5, 14]. LeWM [10], our primary model family, combines this joint-embedding objective with action-conditioned latent prediction. PLDM [11, 12] provides a different joint-embedding dynamics architecture on which we also evaluate ACPC. Related training objectives predict future latent observations from histories and actions in PBL [15], or match predicted and target representations under augmentation in SPR [16]. Subsequent work studies collapse in self-predictive learning [17] and the importance of action conditioning [18]. Theoretical analyses ask when latent prediction retains informative rather than nuisance features [6, 7] or recovers latent state up to a linear transformation [19]. seq-JEPA [20] further studies invariance and equivariance under known view transformations. These works explain how predictive representations and dynamics are learned; we evaluate how a trained predictor responds when its visual input is perturbed.

Robustness to Visual Perturbations. Training-time augmentation improves visual control in DrQ [21] and DrQ-v2 [22], while SODA [23] uses augmentation in an auxiliary representation-learning objective. ViGMO [24] and VIBR [25] learn invariance in latent or value space. ReOI [26] instead modifies observations at test time to reduce distractor sensitivity in visual model-predictive control. Other methods change world-model training. TPC [27] favors temporally predictable information, and DreamerPro [28] replaces pixel reconstruction with prototype prediction. Task Informed Abstractions [29] and Denoised MDPs [30] use task or reward information to separate useful state from distractors. Iso-Dream [31] separates controllable and noncontrollable dynamics, while other approaches combine temporal masking with bisimulation [32], infer the actions of visually similar distractors [33], or use task-aware reconstruction [34]. These methods reflect a broader principle: robustness should remove irrelevant visual variation without discarding distinctions needed for control. Bisimulation formalizes behavioral similarity through rewards and action-conditioned transitions [8, 9, 35], with reward-free variants based only on transitions [36]. Value-equivalent and value-aware representations instead preserve information needed for value prediction or planning [37, 38]. Most closely related, MWM [39] enforces action-conditioned rollout consistency during training. ACPC measures how clean and perturbed latent rollouts diverge in a frozen model under the same action sequence, without retraining it.

Diagnostics for World Models. Model accuracy and control performance do not always move together: one-step likelihood can be a poor guide to downstream performance [40]. This mismatch has motivated methods that connect model learning and evaluation to downstream decisions [41]. World-model evaluation should therefore reflect the intended use of the model, including long rollouts, planning, and policy evaluation [42]. One group of methods tests or enforces whether learned dynamics respect actions. ATM [43] compares action information in encoded and predicted transitions, while Delta-JEPA [44] decodes actions from latent differences. ACID [45] adds inverse-dynamics cycle consistency to the planning cost. Future-Compatible [46] checks whether generated futures agree with their stated actions, and World Models as Group Actions [47] tests identity, inverse, and composition structure. Related diagnostics expose kinematic failures in long rollouts [48] or use frozen inverse-dynamics probes to test whether latents retain action information under visual corruptions [49]. A second group evaluates downstream failure. WMAttack [50] searches for visual attacks on closed-loop world-model agents, while ARB4WM [51] targets their policy, value, and latent dynamics. Other work compares predicted rollouts with environment transitions [52] or compares predicted and true plan costs [53]. Finally, CARRL [54] certifies Q-based action choices under bounded observation uncertainty, and CROP [55] certifies per-state actions and finite-horizon reward bounds through smoothing. These certificates cover policy decisions or return, whereas our bounds preserve only the winner or elite set for an evaluated pair and fixed candidate pool. More broadly, existing diagnostics assess action content, rollout validity, model error, or attack sensitivity. ACPC asks how a task-preserving visual perturbation propagates through a shared-action rollout without requiring the true future.

3 Action-Conditioned Predictive Consistency as a Diagnostic

A visual perturbation can change the trajectory predicted by a world model. ACPC measures this change by rolling a clean history and its perturbed version forward under the same action sequence and comparing the two trajectories. We first define this pairwise measurement. We then show that it bounds perturbation-induced changes in prediction error and candidate cost, which leads to conditions for preserving a planner's selected candidate. To summarize ACPC across a checkpoint, IR captures clean–perturbed pairs with high sensitivity, while SR checks that different states remain separated and prevents a collapsed representation from appearing robust. Finally, we combine IR and SR into an empirical checkpoint screen.

3.1 Pairwise ACPC

Given a clean history and its perturbed version, ACPC compares the rollouts predicted from them under identical actions. Let h denote the clean history and let $\tilde{h}$ be the result of applying the evaluated visual perturbation to h. The frozen encoder maps them to $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both representations are then rolled forward under the action sequence $\mathbf{a} = (a_0, \ldots, a_{H-1})$. Let F_θ denote the frozen action-conditioned predictor. After the first k actions, the two predicted representations are

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \qquad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \tag{1}$$

Here H is the number of autoregressive future steps. ACPC compares predictions in the projected latent space used to evaluate planner costs. Let Π map each predicted representation into this space. To combine all H predicted steps into one trajectory-level distance, we assign nonnegative weights α_k, with $\sum_{k=1}^{H} \alpha_k = 1$, and define the weighted rollout vector

$$\bar{G}_{\mathbf{a}}(z) = \left[\sqrt{\alpha_1}\Pi(\hat{z}_1)^\top \quad \cdots \quad \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top\right]^\top. \tag{2}$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \left\|\bar{G}_{\mathbf{a}}(E_\theta(h)) - \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h}))\right\|_2 = \left(\sum_{k=1}^{H} \alpha_k \left\|\Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k)\right\|_2^2\right)^{1/2}. \tag{3}$$

Low ACPC means that the two predicted trajectories remain close; high ACPC means that they separate. Encoder shift $\|z - \tilde{z}\|_2$ measures the difference before prediction, whereas ACPC measures it after rollout. Using the same action sequence removes action differences from the comparison. ACPC measures rollout consistency, not prediction accuracy.

We use uniform weights $\alpha_k = 1/H$ and $H = 8$, the longest horizon available for every logged window (Appendix C). The planner analysis uses $H = 5$ to match the CEM planning horizon (Section 4.5).

3.2 Prediction-Error Bounds

ACPC itself does not require the observed future. To connect it to prediction error, let $Y_{\mathbf{a}}^H$ denote the observed future under the same action sequence, represented in the same weighted space as the predicted rollouts (Equation (2)). Both rollout errors are measured against this single target. Their difference cannot exceed the distance between the two predictions, which is ACPC.

Proposition 1 (Prediction-error change bound)**.** *Define*

$$e_h = \|\bar{G}_{\mathbf{a}}(E_\theta(h)) - Y_{\mathbf{a}}^H\|_2, \qquad e_{\tilde{h}} = \|\bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) - Y_{\mathbf{a}}^H\|_2.$$

Then, for every paired sample,

$$|e_{\tilde{h}} - e_h| \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{4}$$

In particular, the increase in error caused by the perturbation, $(e_{\tilde{h}} - e_h)_+$, is also bounded by ACPC.

This result follows directly from the reverse triangle inequality; the proof is given in Appendix A. The bound concerns the change in error, not absolute prediction accuracy. Both rollouts can be inaccurate even when ACPC is small. In the experiment, the observed future is used to compute the error change $d = |e_{\tilde{h}} - e_h|$, not ACPC itself (Section 4.4).

3.3 Selection Stability from Planning-Cost Bounds

A planner ranks candidate action sequences by their predicted costs. Changing the input can change each candidate's predicted endpoint and therefore its cost. If these cost changes are too small to close the gaps between candidates, the selected candidate remains unchanged. We now formalize this idea.

For each candidate action sequence $\mathbf{a}^j$, we roll both inputs forward under that sequence. Let

$$x_j = \Pi(F_\theta^H(E_\theta(h), \mathbf{a}^j)), \qquad \tilde{x}_j = \Pi(F_\theta^H(E_\theta(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is one component of the candidate-specific ACPC. Whenever $\alpha_H > 0$,

$$r_j \leq \frac{\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)}{\sqrt{\alpha_H}}.$$

Proposition 2 (Planning-cost bounds)**.** *Suppose the planner scores candidate j by its squared distance to a fixed goal embedding g:*

$$C_j = \|x_j - g\|_2^2, \qquad \tilde{C}_j = \|\tilde{x}_j - g\|_2^2.$$

Define the candidate-specific cost-change bound

$$b_j = r_j\big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big). \tag{5}$$

Then $|\tilde{C}_j - C_j| \leq b_j$.

Let w and $\tilde{w}$ be the winners under the clean and perturbed inputs. If the winner changes, the perturbed winner's excess cost under the clean prediction satisfies

$$0 \leq C_{\tilde{w}} - C_w \leq b_{\tilde{w}} + b_w. \tag{6}$$

The bound b_j is computed from the evaluated endpoints and requires no global Lipschitz constant. If the clean winner w leads every competitor j by more than $b_w + b_j$, the perturbation cannot reverse their order. The same argument preserves the top-k elite set when every clean elite–non-elite gap exceeds the corresponding pair of bounds. Appendix A gives the exact conditions (Proposition 3).

Because r_j is bounded by candidate-specific ACPC, substituting its bound into b_j gives

$$|\tilde{C}_j - C_j| \;\leq\; b_j \;\leq\; \frac{\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)}{\sqrt{\alpha_H}}\big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big), \tag{7}$$

Thus, ACPC bounds how much each candidate's cost can change, while the clean cost gaps determine whether that change can alter the planner's selection. These certificates require evaluating every candidate under both inputs, so they support post-hoc analysis rather than online screening.

CEM plans in rounds. In each round, it scores a set of candidate action sequences, keeps the best group (the elite set), and uses that group to generate the candidates for the next round [13]. We compare a clean and perturbed run that start from the same proposal and use the same random samples. If the perturbation does not change the elite set in any round, both runs generate the same candidates in the next round and eventually return the same action (Corollary 1). If the elite set changes, the guarantee no longer applies. This result concerns the planner's model-based choice, not its return in the environment.

3.4 Checkpoint-Level IR and SR

Pairwise ACPC measures one clean–perturbed pair under one action sequence. Evaluating a checkpoint requires summarizing this measurement across many histories. The *Invariance Radius* (IR) measures how sensitive these paired rollouts are to the visual perturbation; lower IR is better. Low IR alone is not enough because a collapsed representation would make every rollout look similar. The *Separation Rate* (SR) therefore checks whether different states remain distinguishable after rollout; higher SR is better. A favorable checkpoint should have both low IR and high SR.

To compute IR, we select n logged history windows as anchors. Anchor i provides a clean history h_i and its recorded action sequence $\mathbf{a}_i = (a_{i,0}, \ldots, a_{i,H-1})$. We apply the visual perturbation M times to obtain $\tilde{h}_i^{(1)}, \ldots, \tilde{h}_i^{(M)}$, and compute ACPC for each clean–perturbed pair under the same recorded actions.

Raw latent distances can have different scales across histories. We therefore divide each ACPC value by the anchor's typical one-step clean motion, denoted by s_i. Specifically, s_i is the median projected displacement between consecutive clean observations in the anchor window (Equation (23) and appendix C). This expresses ACPC relative to the amount of motion normally seen in that history. With a small numerical stabilizer $\varepsilon_{\text{norm}}$, define

$$R_i^{(m)} = \frac{\text{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\text{norm}}}, \qquad \bar{R}_i = \frac{1}{M}\sum_{m=1}^{M} R_i^{(m)}. \tag{8}$$

The average $\bar{R}_i$ gives one normalized sensitivity value for each anchor. Let Q_q denote the empirical q-quantile across anchors. The raw IR is

$$\text{IR}_q^{\text{raw}}(\theta) = Q_q\big(\{\bar{R}_i\}_{i=1}^n\big). \tag{9}$$

IR is one value for the checkpoint. We use $q = 0.90$ so that IR reflects anchors with relatively high sensitivity, which a mean could hide. The result is stable for q between 0.80 and 0.95 and for the tested rollout horizons (Table 1).

IR checks whether perturbed views remain close to their clean counterparts. SR checks whether this invariance also preserves differences between states. For each eligible anchor, the protocol selects a nearby logged history with a different state-coordinate label. Both histories are rolled forward under the anchor's recorded actions, so their separation is not caused by different action sequences. Their trajectory distance is normalized by the same clean-motion scale used for IR and is denoted by D_i^{diff} (Equation (24) and appendix C).

Let $\mathcal{I}$ be the set of anchors for which such a comparison is available. SR is the fraction whose different-state distance exceeds raw IR by a margin δ:

$$\text{SR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \mathbf{1}\big[D_i^{\text{diff}} > \text{IR}_q^{\text{raw}}(\theta) + \delta\big], \tag{10}$$

We use $q = 0.90$ and $\delta = 0.10$. SR therefore reports how often a tested different-label pair remains beyond the checkpoint's perturbation radius plus the fixed margin.

A collapsed representation makes IR small, but it also makes every D_i^{diff} zero and therefore fails SR. Conversely, a checkpoint can have high SR while remaining sensitive to visual perturbations, so IR is still needed. SR applies only to the state-coordinate labels used in the tested pairs. Appendix C gives the exact pairing rules and eligible-pair counts.

3.5 Checkpoint Screening

SR uses raw IR because it compares distances within the same checkpoint. To compare a trained checkpoint with its unaugmented reference, we use relative IR. Let θ_0 denote the unaugmented checkpoint from the same task, training run, and model family. We define

$$\text{IR}_q^{\text{rel}}(\theta;\theta_0) = \frac{\text{IR}_q^{\text{raw}}(\theta)}{\text{IR}_q^{\text{raw}}(\theta_0)}. \tag{11}$$

Relative IR equals 1 for the reference checkpoint. Values below 1 indicate lower sensitivity than the reference, while values above 1 indicate higher sensitivity. This comparison is defined within the same task, training run, and model family; it does not make IR directly comparable across them.

A checkpoint passes the screen only when relative IR is below its threshold and SR is above its threshold. We combine these two requirements into one score. Each term measures the checkpoint's normalized margin from one threshold, and the minimum selects the weaker of the two results.

For model family $\mathcal{F}$ and visual shift υ, let the thresholds be t_{IR} and t_{SR}. All measurements in a score use the same shift. With a small numerical constant $\varepsilon_{\text{score}}$, define

$$S_{\mathcal{F},\upsilon}(\theta;\theta_0) = \min\left\{\frac{t_{\text{IR}} - \text{IR}_q^{\text{rel}}(\theta;\theta_0)}{|t_{\text{IR}}| + \varepsilon_{\text{score}}}, \frac{\text{SR}_{q,\delta}(\theta) - t_{\text{SR}}}{|t_{\text{SR}}| + \varepsilon_{\text{score}}}\right\}. \tag{12}$$

The score is nonnegative exactly when both requirements pass. A negative score means that at least one requirement fails.

We choose the two thresholds using planning success on source tasks and apply them unchanged to held-out tasks. Thresholds are selected separately for each model family. Once fixed, the score requires no task-success labels.

To compare checkpoint θ_1 with its reference θ_0, we report the change in screening score: $\Delta S_{\mathcal{F},\upsilon}(\theta_1;\theta_0) = S_{\mathcal{F},\upsilon}(\theta_1;\theta_0) - S_{\mathcal{F},\upsilon}(\theta_0;\theta_0)$. A positive ΔS means that θ_1 receives a better joint IR–SR score than the reference. It does not label either checkpoint as robust.

The experiments evaluate the method at two levels. Pair-level experiments test whether ACPC reflects changes in prediction error and planner selection (Sections 4.4 and 4.5). Checkpoint-level experiments test whether a fixed IR–SR screen transfers across tasks, model families, and visual shifts (Sections 4.6 to 4.8). The checkpoint result is an empirical screening claim; it does not follow from the pairwise bounds.

4 Experiments

After describing the evaluation protocol, we evaluate the diagnostic at two levels. At the pair level, we test whether ACPC reflects prediction-error change and CEM selection regret. At the checkpoint level, we examine how IR and SR vary across recovery and whether checkpoint screening transfers across tasks, PLDM, blur, and resize. We begin with a local-geometry case study to visualize the representation pattern behind these measurements.

4.1 Evaluation Protocol

Models, Tasks, and Evaluation. Most experiments use LeWM [10]. We also evaluate PLDM [11, 12] to test the diagnostic on a second world-model architecture. We use four control tasks: TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube (Cube) 3D manipulation. For planning evaluation, CEM uses the frozen world model to optimize five-step action sequences. We report the *planning success rate*, defined as the percentage of episodes that reach the task goal.

We use *visual perturbation* for one transformed history and *visual shift* for the distribution of such transformations. The main experiments add Gaussian noise with $\sigma = 0.08$ to the history observations while keeping the goal image clean. Additional experiments use Gaussian blur ($k = 15$) and resize (scale 0.25).

For each task and model family, the Gaussian-noise sweep contains nine training conditions: one without noise augmentation and eight with full-sequence Gaussian-noise augmentation at $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$. Both LeWM and PLDM have three independent training runs for each task–condition pair (seeds 3072/3073/3074). Each trained checkpoint is evaluated with seeds 42/43/44, using 100 episodes per evaluation seed. We use *task–run cell* for one task and one training run.

The training run is the unit of replication. The three evaluation seeds measure only within-run variability, so the reported means and dispersions across runs are descriptive rather than population estimates.

Diagnostic Protocol. IR uses 100 logged anchors, five Gaussian-noise draws at the evaluation severity $\sigma = 0.08$ (blur and resize diagnostics instead apply the corresponding shift), eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. To compute SR, the protocol selects, for each anchor, a nearby history with a different endpoint-state label and checks whether the two rollouts remain farther apart than raw IR plus 0.10. Appendix C gives the exact pairing, normalization, and threshold rules. Checkpoint comparisons use relative IR from Equation (11).

Threshold Protocol. A checkpoint in the Gaussian-noise training sweep meets the *success-rate criterion* if it recovers at least 80% of the improvement from the unaugmented checkpoint to the best checkpoint at evaluation noise $\sigma = 0.08$, while losing no more than five percentage points on clean observations. Both constants were fixed a priori. We select $(t_{\mathrm{IR}}, t_{\mathrm{SR}})$ on every nonempty proper subset of the four tasks and apply the thresholds unchanged to the remaining tasks. The one-, two-, and three-task selection settings produce $4 + 6 + 4 = 14$ directional threshold-selection/test partitions. Metrics first average training runs within each test task and then weight tasks equally; checkpoint rows are never treated as independent replicates. Success rates are used to select and evaluate thresholds, but they are not inputs to ACPC, IR, or SR.

For blur and resize, each task uses the thresholds selected under Gaussian noise on the other three tasks. We compare 24 checkpoint pairs (four tasks $\times$ three runs $\times$ two shifts). Each pair contains the unaugmented

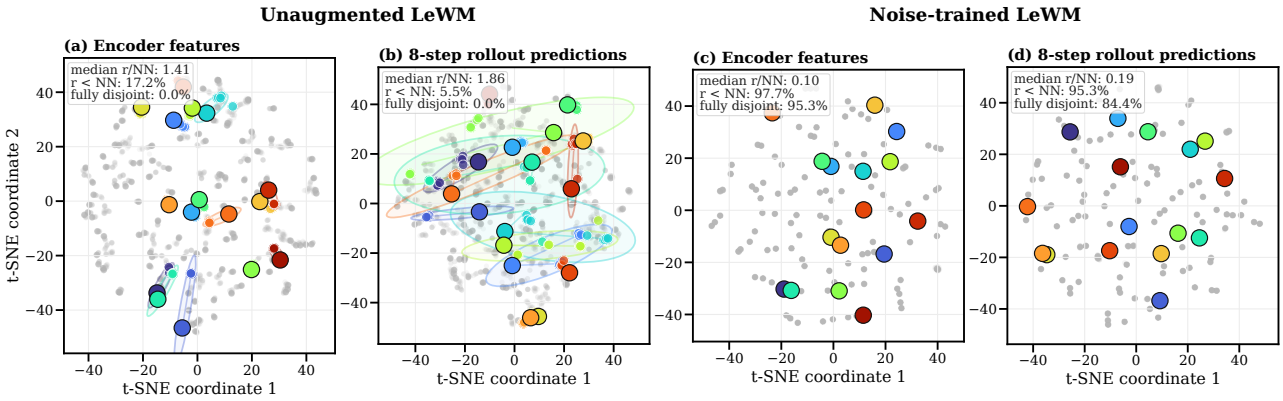

Figure 2: Descriptive local-geometry case study motivating the controlled ACPC comparison. Panels (a)–(d) compare a matched pair of PushT checkpoints—the left two unaugmented, the right two trained with Gaussian-noise augmentation at $\sigma_{\max} = 0.08$—at the encoder output and after eight autoregressive rollout steps. Each color denotes one of 16 highlighted histories: the large black-rimmed marker is its clean anchor, smaller same-color markers are its 18 perturbed views, and the faint ellipse is their 90% t-SNE envelope; gray points show all other histories and views. Under strong contraction, smaller markers and ellipses can lie beneath the clean marker. Each panel's upper-left summary reports, across all 128 anchors, the median r/NN and the percentages with $r < \mathrm{NN}$ and with fully disjoint high-dimensional enclosing balls. The t-SNE coordinates and ellipses are qualitative and enter none of these metrics.

checkpoint and the checkpoint trained at $\sigma_{\max} = 0.08$. A pair is positive when success under the shift improves by at least five percentage points and clean success decreases by no more than five points. We make no blur- or resize-specific adjustment. This analysis also does not benchmark alternative signals such as encoder shift or one-step ACPC.

4.2 Local Geometry of Perturbed Views

We begin with a PushT case study showing how visual noise can make perturbed views of one history overlap with other histories. For one clean history and its perturbed views, the sampled visual radius r measures the spread of the perturbation cloud; nearest-clean spacing NN is the distance to the nearest other clean history. The ratio r/NN asks whether visual variation overwhelms local between-history spacing, while a symmetric two-ball test counts fully disjoint clouds. These quantities are descriptive: neighbors are chosen in learned space, each clean rollout follows its own recorded actions, and a collapsed representation makes every radius small. They motivate the controlled ACPC comparison but do not replace it.

Using a fixed PushT case study, we compare a matched pair of LeWM checkpoints: one unaugmented and one trained with full-sequence Gaussian-noise augmentation at $\sigma_{\max} = 0.08$. We inspect both the encoder output and the representation after eight autoregressive rollout steps. All ratios and fractions use the original 192-dimensional projected latent space in which the planner computes costs; formal definitions and sampling details are in Appendix B. The t-SNE visualization [56] is qualitative only.

Figure 2 summarizes the case study. Without augmentation, the median r/NN is 1.41 at the encoder and 1.86 after eight rollout steps, and none of the 128 anchor clouds is fully disjoint. After Gaussian-noise augmentation, the corresponding ratios fall to 0.10 and 0.19; the fully disjoint fractions rise to 95.3% at the encoder and 84.4% after the rollout. Thus, in this fixed case study, augmentation makes perturbed views of the same history substantially more compact relative to nearby clean histories. Much of this separation remains after prediction.

This case study shows that augmentation can contract perturbation-induced variation relative to nearby clean histories, including after prediction. It does not establish task-relevant separation or planning robustness. We therefore turn to checkpoint-level IR and SR and their relationship with planning performance across the complete augmentation sweep (Figure 3).

4.3 IR and SR across Checkpoint Recovery

At evaluation noise $\sigma = 0.08$, the unaugmented LeWM checkpoints lose 25.9 ± 2.4, 74.6 ± 5.6, 41.0 ± 1.4, and 22.1 ± 2.8 percentage points in success rate on TwoRoom, PushT, Reacher, and Cube, respectively. Gaussian-noise augmentation recovers performance over a range of training levels whose location differs by task. Figure 3

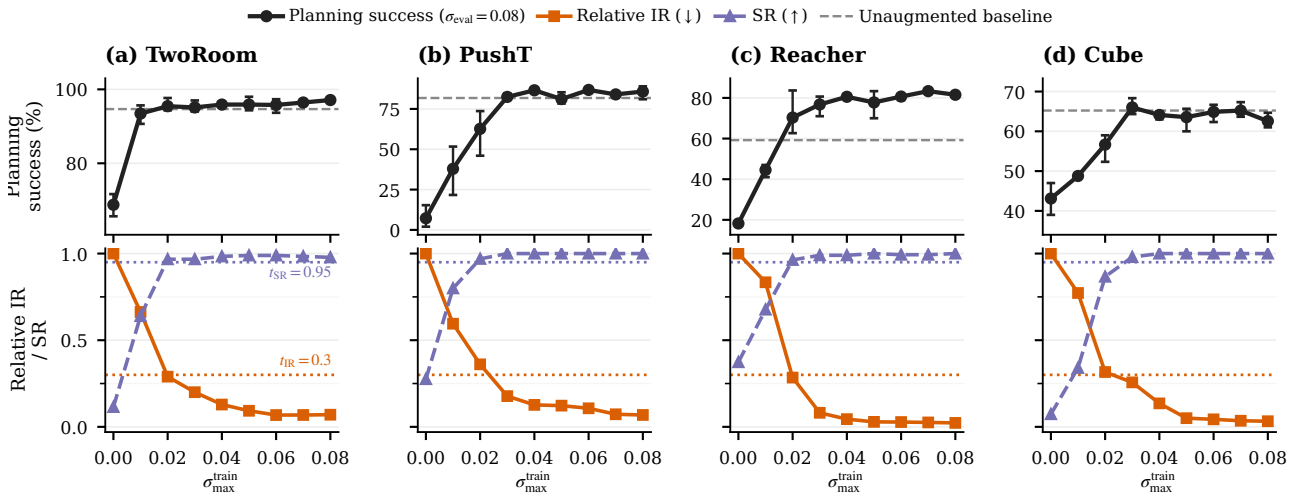

Figure 3: Checkpoint-level planning performance and diagnostics across the complete Gaussian-noise training sweep. We report planning success rate at evaluation noise $\sigma = 0.08$, relative IR, and SR. Points are across-run means; success-rate error bars span the three training runs, and success-rate axes are scaled per task. The dashed line is the clean success rate of the unaugmented checkpoint; the leftmost black point is that checkpoint evaluated at $\sigma = 0.08$. Relative IR equals one there by construction. Lower IR and higher SR are favorable. Dotted lines mark $t_{\text{IR}} = 0.3$ and $t_{\text{SR}} = 0.95$ (Section 4.6).

compares success rate with IR and SR across the complete sweep.

On Reacher, augmentation also raises clean success from 59.2% to 76–82%. Its recovery under noisy evaluation therefore cannot be attributed to robustness alone; the checkpoints also improve on clean observations.

Every augmentation level that meets the success-rate criterion has lower IR and higher SR than its unaugmented reference, although neither measure is monotone at every level. Across the 108 LeWM checkpoint rows, 77 pass the reported IR threshold $t_{\text{IR}} = 0.3$, and all 77 also pass $t_{\text{SR}} = 0.95$. Thus every accepted checkpoint combines reduced same-history sensitivity with retention of the evaluated state-coordinate distinctions. The different-label distance medians are largely stable across augmentation levels, so the rise in SR mainly reflects contraction of the same-history radius rather than expansion of different-label distances.

IR tracks the recovery region, while SR verifies that the sampled state-coordinate distinctions remain outside the contracted radius. A first-order estimate of the composed encoder–rollout Jacobian offers one mechanism for the IR reduction (Appendix G). The next two experiments test whether ACPC uses recorded-action information and whether it tracks a planner-facing quantity (Sections 4.4 and 4.5).

4.4 ACPC and Prediction-Error Change

Does ACPC help predict how much a visual perturbation changes multi-step prediction error? For each history pair, Proposition 1 shows that the two errors against the same logged future can differ by at most ACPC_H. We test whether measured ACPC is informative about this bounded quantity, the error drift $d = |e_{\tilde{h}} - e_h|$. We evaluate at the protocol horizon $H = 8$, the longest horizon with a complete logged common future for every anchor; Table 1 shows that the checkpoint-level conclusions are stable for $H \in \{1, 2, 4, 8\}$.

Data and protocol. The analysis uses the unaugmented checkpoint of each task and training run. Trajectories are partitioned into 16 disjoint groups. Each history pair contributes rows at Gaussian-noise standard deviations $\sigma \in \{0.02, 0.05, 0.08\}$ with two perturbation draws; zero-severity probes serve only as identity checks. A ridge model is fitted on 15 groups and evaluated on the remaining group, rotating through all 16, and all rows from one trajectory group stay together. We report the mean absolute error (MAE) on groups excluded from model fitting.

Three nested regressions. Every model contains the probe severity and the encoder-history distance $\|z - \tilde{z}\|_2$.

- **Base** adds one-step ACPC under the recorded action: the information available without a multi-step rollout.
- **Base+Control$_8$** adds the strongest *destroyed-action control*: eight-step ACPC recomputed with *zeroed* actions (every action set to zero), *swapped* actions (the recorded actions of a different trajectory), or *shuffled*

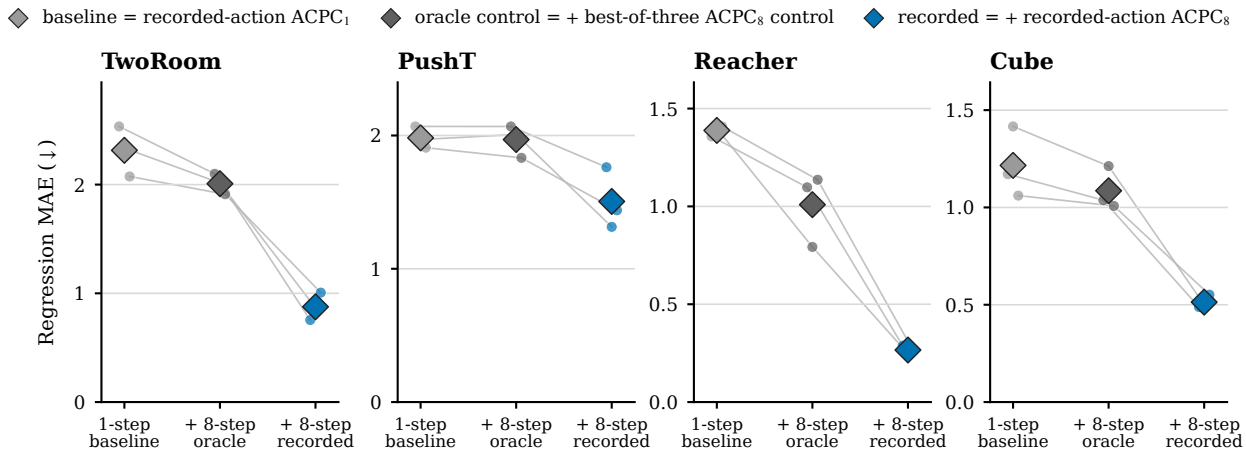

Figure 4: Cross-validated MAE for predicting the error drift d on the unaugmented checkpoints; lower is better. Each model is evaluated on trajectory groups excluded from fitting. Small points are training runs, thin lines join the same run, and diamonds are task means. The in-figure legend labels correspond to Base, Base+Control$_8$, and Base+ACPC$_8$ as defined in Section 4.4. Base+ACPC$_8$ is lowest in all 12 task–run cells; latent-error scales are task-specific, so compare within a panel.

actions (the anchor's own actions in permuted temporal order). Each control performs the same eight-step rollout, so it carries rollout length without the recorded action sequence. "Strongest" is an oracle choice: in each task–run cell we report whichever of the three controls attains the lowest test MAE, which makes the comparison conservative.

- **Base+ACPC$_8$** instead adds eight-step ACPC under the recorded actions—the only feature whose action sequence is the one that generated the logged future, and hence the feature matching the right-hand side of Proposition 1.

All eight-step features use uniform weights $\alpha_k = 1/8$ in Equation (3). If Base+ACPC$_8$ improves on Base, multi-step information helps; if it also improves on Base+Control$_8$, the gain is attributable to the recorded actions rather than to merely rolling out eight steps.

Results. Base+ACPC$_8$ attains the lowest cross-validated MAE in all 12 task–run cells (Figure 4). For each training run we average the four task-level relative MAE reductions equally and report the mean ± sample standard deviation across the three training runs: the reduction is $55.9 \pm 4.7\%$ relative to Base and $51.3 \pm 3.5\%$ relative to Base+Control$_8$. These results concern prediction of the error drift; they do not compare the absolute forecast accuracy of one-step and eight-step world models. Appendix D reports the task-level values and selected controls (Tables 4 and 5). Eight-step ACPC carries information about the bounded error change that neither encoder shift, one-step ACPC, nor any same-horizon destroyed-action control provides.

4.5 ACPC and CEM Selection Regret

A prediction change matters to planning only if it affects the plan that CEM selects. The fixed-pool bound motivates the quantity below, but adaptive CEM can generate different later pools once the two elite sets diverge. We therefore evaluate full paired CEM runs rather than treating the fixed-pool condition as a run-level certificate. Our downstream quantity is *clean-reference selection regret*. Let $\mathbf{a}$ and $\tilde{\mathbf{a}}$ be the final plans selected from the clean and perturbed histories, respectively. We measure $(C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a}))_+$: the extra clean-history model cost incurred by selecting the perturbed-history plan. We refer to this single-decision quantity as *CEM selection regret*. It uses the model's squared latent goal cost; it is neither cumulative regret nor simulator return.

The paired CEM branches follow Appendix H: same initial proposal, common random numbers, each branch updating from its own elites. For every candidate, we compute one- and five-step ACPC along its action sequence and take the pool q90; five steps match the planner's prediction horizon. We compare two ridge regressions. The baseline uses perturbation severity, training condition, candidate-level one-step ACPC q90, and the clean gap between the best and second-best candidates. The expanded model adds candidate-level five-step ACPC q90. Within each training run, both regressions are fitted on three tasks and tested on the remaining task, rotating

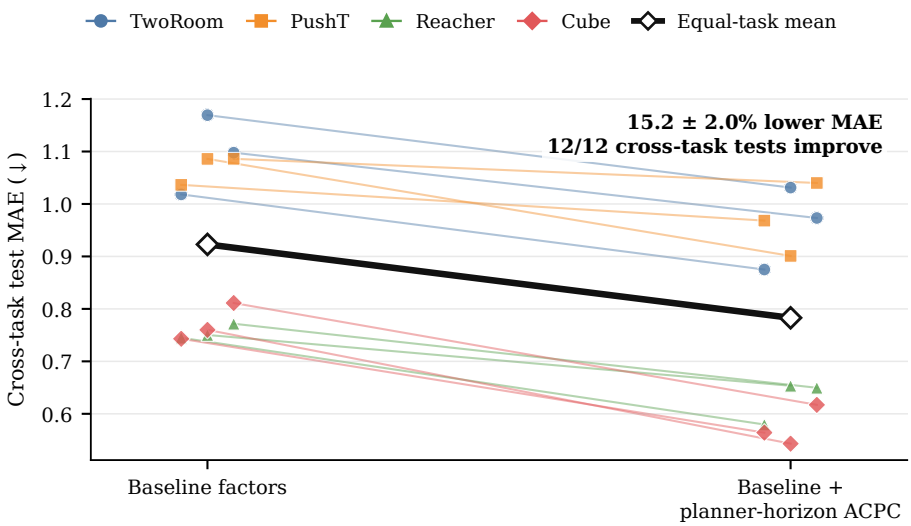

Figure 5: Adding planner-horizon ACPC improves prediction of adaptive-CEM selection regret on tasks excluded from regression fitting. Each colored line connects the baseline and expanded-model errors for one test task in one training run; the thick black line is the equal-task average across the three runs. The added feature is candidate-specific ACPC q90 at the planner's five-step prediction horizon. The baseline already contains severity, training condition, one-step ACPC q90, and the clean best–second-best cost gap. MAE is computed on $\log(1 + \text{selection regret})$.

which task is excluded from fitting. We evaluate MAE on $\log(1 + \text{selection regret})$; Appendix H gives the CEM budget and sample counts.

Adding planner-horizon ACPC lowers cross-task test MAE by $15.2 \pm 2.0\%$ (mean $\pm$ sample standard deviation across training runs). The error decreases in all 12 task–run test cases (Figure 5). Since the two regressions differ only by the planner-horizon feature, this gain shows that the planner-horizon rollout carries additional cross-task predictive information. The comparison does not separate action conditioning from horizon length; it tests what the five-step ACPC feature adds. Candidate-specific ACPC therefore helps predict the clean-model cost of a perturbation-induced CEM selection change across tasks. It does not show that ACPC changes the selected action or improves environment performance.

4.6 Checkpoint Screening across Tasks

Because both the augmentation sweep and the threshold grid are discrete, we test for a useful shared threshold range rather than an exact universal boundary. A decision is correct when it matches the success-rate criterion of Section 4.1. We also count the grid steps between the first accepted checkpoint and the first checkpoint that meets that criterion.

Across the 14 source/test splits, 13 choose $(t_{\text{IR}}, t_{\text{SR}}) = (0.3, 0.95)$; only the Reacher-only split chooses $(0.1, 0.95)$. With two or three source tasks, the first accepted checkpoint is within 0.5 grid steps of recovery on average and never differs by more than one step (balanced accuracy 0.900, precision/recall 0.913/0.953). Single-source selection is less reliable: balanced accuracy averages 0.855 and ranges from 0.723 to 0.913. Reacher alone chooses the tighter IR threshold and delays acceptance on the other tasks by as many as five levels (Appendix E).

The predominant rule accepts a checkpoint only when relative IR is at most 0.3 and SR is at least 0.95. IR limits same-history sensitivity, while SR requires the tested state-coordinate distinctions to remain separated. Across the sweep, both measures improve over augmentation levels associated with recovery (Figure 3). Because $t_{\text{IR}} = 0.3$ is the largest tested value, the upper edge of the useful range remains unresolved.

4.7 Checkpoint Screening for PLDM

We apply the same ACPC, IR, and SR definitions to PLDM, which uses a different architecture and training recipe. The experiment repeats the nine-checkpoint Gaussian-noise training sweep, paired histories, eight-step rollouts, success-rate criterion, and threshold-selection procedure across three independent training runs. Threshold selection stays within PLDM: each test task uses the other three PLDM tasks as sources.

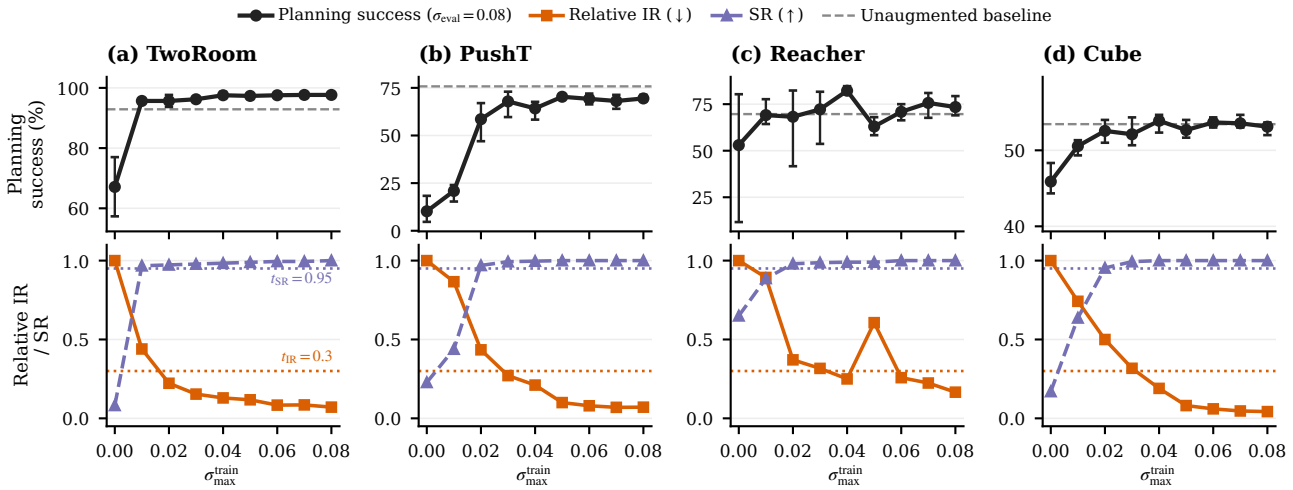

Figure 6: PLDM Gaussian-noise training sweep over three independent training runs. Curves show run means, and success-rate error bars span the run range. The dashed gray line marks clean success of the unaugmented checkpoint, and the leftmost black point is its success at evaluation noise $\sigma = 0.08$. Relative IR equals one at that checkpoint. Dotted lines mark $t_{\mathrm{IR}} = 0.3$ and $t_{\mathrm{SR}} = 0.95$. Across tasks, stronger augmentation generally lowers relative IR and raises SR, while planning recovery varies across training runs.

Three of the four leave-one-task-out selections choose $(t_{\mathrm{IR}}, t_{\mathrm{SR}}) = (0.3, 0.95)$; the split tested on TwoRoom chooses $(0.2, 0.95)$. Across the 108 checkpoint decisions, balanced accuracy is 0.681, with precision 0.667 and recall 0.786. Averaging the four task-level balanced accuracies gives 0.699. Screening is strongest on TwoRoom (0.875) and less reliable on PushT (0.571), Reacher (0.632), and Cube (0.717).

The same low-IR, high-SR pattern appears across the PLDM sweeps, but its agreement with recovery is weaker than for LeWM. This result supports the use of IR and SR with a second architecture while showing that their checkpoint-screening accuracy is model- and task-dependent.

4.8 Checkpoint Comparison under Blur and Resize

Blur ($k = 15$) and resize (scale 0.25) each provide one fixed-severity check outside the Gaussian-noise shift used for threshold selection. For each of the 24 checkpoint pairs in Section 4.1, we recompute IR and SR under the new shift and compare the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart using ΔS. A positive ΔS means that the augmented checkpoint gets a higher joint IR–SR score. This is a relative comparison, not an absolute pass decision. No blur or resize outcome enters threshold selection.

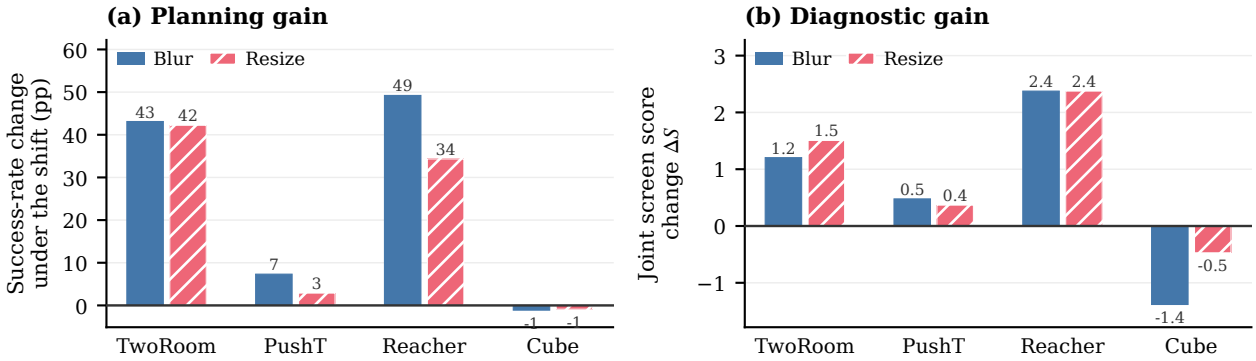

Figure 7: Relative checkpoint comparison under blur and resize. Bars average the three training runs for each task and shift; per-pair values are in Table 9. Panel (a) shows the change in planning success from the unaugmented to the $\sigma_{\max} = 0.08$ checkpoint. Panel (b) shows the change in the IR–SR score of Equation (12); positive favors the augmented checkpoint but is not an absolute pass decision.

The sign of ΔS agrees with whether the augmented checkpoint meets the predefined success criterion on

22 of 24 pairs (balanced accuracy 0.889), and its magnitude has Spearman correlation 0.835 with the success change. The two disagreements are PushT pairs that improve by four percentage points, one point below the criterion. Because ΔS compares two checkpoints, these fixed-severity results support relative ordering, not an absolute robustness decision.

5 Discussion and limitations

What is supported. The strongest evidence is pair-level. Recorded-action ACPC predicts multi-step error drift beyond encoder shift, one-step ACPC, and same-horizon destroyed-action controls. Planner-horizon ACPC also improves prediction of CEM selection regret beyond the corresponding one-step feature and clean candidate margin. These results align with the exact samplewise bounds while testing quantities that remain informative beyond simpler controls.

At the checkpoint level, relative IR tracks recovery across the Gaussian-noise training sweeps, and thresholds chosen on source tasks remain useful on held-out tasks. Under blur and resize, the joint score agrees with the predefined success criterion on 22 of 24 pairs.

Why both measures are necessary. IR alone rewards any contraction, including collapse or excessive invariance. SR guards against that failure by requiring selected histories with different state-coordinate labels to remain separated. Conversely, SR alone does not require two views of the same history to agree. The two measures therefore answer different questions and must be used together. Their scope is limited to the tested visual changes and state-coordinate labels.

Other limits concern the downstream claims. The planner experiment predicts a clean-model cost; it does not intervene on CEM or show improved simulator return. The adaptive-CEM certificate applies only while candidate pools remain aligned and stops at the first failed elite condition. The selected IR point lies at the upper edge of the evaluated grid. Blur and resize are each tested at one severity. Finally, the main recovery sweep varies one training intervention: Gaussian-noise augmentation.

Future work. Important next steps are direct task outcomes for ACPC-informed planner interventions, multiple severities per visual shift, richer pairs with different state labels, additional training interventions, and more JEPA-family world models.

6 Conclusion

ACPC measures how much a state-preserving visual change alters a world model's predicted trajectory when the actions are held fixed. Exact inequalities relate this change to prediction-error drift and planner candidate-cost movement. IR summarizes upper-tail same-history sensitivity, while SR checks that nearby histories with different state-coordinate labels remain separated. In our experiments, recorded-action ACPC improves prediction of error drift and CEM selection regret, and joint IR–SR thresholds chosen on source tasks identify recovery on held-out tasks. Results on PLDM, blur, and resize provide initial evidence beyond the primary Gaussian-noise setting.

References

[1] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[2] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[3] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022. Version 0.9.2.

[4] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

[6] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint-embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 21897–21937, 2025.

[7] Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 91300–91336, 2024.

[8] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[9] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[10] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[11] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. arXiv preprint arXiv:2211.10831, 2022. Self-Supervised Learning: Theory and Practice Workshop at NeurIPS 2022.

[12] Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 43905–43941, 2025.

[13] Dirk P. Kroese, Sergey Porotsky, and Reuven Y. Rubinstein. The cross-entropy method for continuous multi-extremal optimization. *Methodology and Computing in Applied Probability*, 8(3):383–407, 2006.

[14] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[15] Zhaohan Daniel Guo, Bernardo Avila Pires, Bilal Piot, Jean-Bastien Grill, Florent Altché, Rémi Munos, and Mohammad Gheshlaghi Azar. Bootstrap latent-predictive representations for multitask reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3875–3886. PMLR, 2020.

[16] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[17] Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Avila Pires, Yash Chandak, Rémi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, András György, Shantanu Thakoor, Will Dabney, Bilal Piot, Daniele Calandriello, and Michal Valko. Understanding self-predictive learning for reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 33632–33656. PMLR, 2023.

[18] Khimya Khetarpal, Zhaohan Daniel Guo, Bernardo Avila Pires, Yunhao Tang, Clare Lyle, Mark Rowland, Nicolas Heess, Diana L. Borsa, Arthur Guez, and Will Dabney. A unifying framework for action-conditional self-predictive reinforcement learning. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 181–189. PMLR, 2025.

[19] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model? arXiv preprint arXiv:2605.26379, 2026.

[20] Hafez Ghaemi, Eilif B. Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 32943–32973, 2025.

[21] Denis Yarats, Ilya Kostrikov, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[22] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[23] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.

[24] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[25] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[26] Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.

[27] Tung D. Nguyen, Rui Shu, Tuan Pham, Hung Bui, and Stefano Ermon. Temporal predictive coding for model-based planning in latent space. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8130–8139. PMLR, 2021.

[28] Fei Deng, Ingook Jang, and Sungjin Ahn. DreamerPro: Reconstruction-free model-based reinforcement learning with prototypical representations. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 4956–4975. PMLR, 2022.

[29] Xiang Fu, Ge Yang, Pulkit Agrawal, and Tommi Jaakkola. Learning task informed abstractions. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 3480–3491. PMLR, 2021.

[30] Tongzhou Wang, Simon S. Du, Antonio Torralba, Phillip Isola, Amy Zhang, and Yuandong Tian. Denoised MDPs: Learning world models better than the world itself. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 22591–22612. PMLR, 2022.

[31] Minting Pan, Xiangming Zhu, Yunbo Wang, and Xiaokang Yang. Iso-Dream: Isolating and leveraging noncontrollable visual dynamics in world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 23178–23191, 2022.

[32] Ruixiang Sun, Hongyu Zang, Xin Li, and Riashat Islam. Learning latent dynamic robust representations for world models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47234–47260. PMLR, 2024.

[33] Yucen Wang, Shenghua Wan, Le Gan, Shuai Feng, and De-Chuan Zhan. AD3: Implicit action is the key for world models to distinguish the diverse visual distractors. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51546–51568. PMLR, 2024.

[34] Miles Hutson, Isaac Kauvar, and Nick Haber. Policy-shaped prediction: Avoiding distractions in model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 13124–13148, 2024.

[35] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[36] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[37] Christopher Grimm, André Barreto, Satinder P. Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5541–5552, 2020.

[38] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[39] Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction. arXiv preprint arXiv:2603.07799, 2026.

[40] Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Proceedings of the 2nd Conference on Learning for Dynamics and Control*, volume 120 of *Proceedings of Machine Learning Research*, pages 761–770. PMLR, 2020.

[41] Ran Wei, Nathan Lambert, Anthony D. McDonald, Alfredo Garcia, and Roberto Calandra. A unified view on solving objective mismatch in model-based reinforcement learning. *Transactions on Machine Learning Research*, 2024.

[42] Yang Yu, Shiyuan Zhang, Yifei Sheng, Haoxiang Ren, and Haoxin Lin. How should world models be evaluated for embodied decision-making? a decision-making-centric position. *arXiv preprint arXiv:2606.15032*, 2026.

[43] Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models. arXiv preprint arXiv:2606.09028, 2026.

[44] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding. arXiv preprint arXiv:2606.31232, 2026.

[45] Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models. arXiv preprint arXiv:2607.02403, 2026.

[46] Bo-Kai Ruan, Teng-Fang Hsiao, Ling Lo, and Hong-Han Shuai. Is the future compatible? Diagnosing dynamic consistency in world action models. arXiv preprint arXiv:2605.07514, 2026.

[47] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions. arXiv preprint arXiv:2605.24578, 2026.

[48] Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure. arXiv preprint arXiv:2607.05966, 2026.

[49] Jewon Yeom, Hanseul Kim, Jeongjae Park, Sungmok Jung, Jaejin Lee, and Taesup Kim. What makes video world model latents action-relevant: Prediction over reconstruction. *arXiv preprint arXiv:2606.07687*, 2026.

[50] Zhixiang Guo, Siyuan Liang, Shi Fu, Cheng Guo, András Balogh, Márk Jelasity, and Dacheng Tao. WMAttack: Automated attack search for adversarial evaluation of world-model agents. *arXiv preprint arXiv:2605.23220*, 2026.

[51] Junjian Zhang, Hao Tan, Ruonan Li, Dong Zhu, Aiping Li, and Zhaoquan Gu. ARB4WM: An adversarial robustness benchmark for world models in continuous control. *arXiv preprint arXiv:2606.16605*, 2026.

[52] Donna Vakalis. Operator-on-F complements value-equivalence: A planning-time diagnostic for latent world models. *arXiv preprint arXiv:2607.04464*, 2026.

[53] Hanzhe You, Yonggang Zhang, Maohao Ran, Zhiqin Yang, Zhenyuan Zhang, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. A control theory of predictability in latent world models. *arXiv preprint arXiv:2607.10362*, 2026.

[54] Björn Lütjens, Michael Everett, and Jonathan P. How. Certified adversarial robustness for deep reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 1328–1337. PMLR, 2020.

[55] Fan Wu, Linyi Li, Zijian Huang, Yevgeniy Vorobeychik, Ding Zhao, and Bo Li. CROP: Certifying robust policies for reinforcement learning through functional smoothing. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[56] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.

[57] Salah Rifai, Pascal Vincent, Xavier Muller, Xavier Glorot, and Yoshua Bengio. Contractive auto-encoders: Explicit invariance during feature extraction. In Lise Getoor and Tobias Scheffer, editors, *Proceedings of the 28th International Conference on Machine Learning*, pages 833–840. Omnipress, 2011.

[58] Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 18(3):1059–1076, 1989.

A Proofs and Fixed-Pool Analysis

Proof of Proposition 1. Write $u = \bar{G}_{\mathbf{a}}(E_\theta(h))$, $v = \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h}))$, and $y = Y^H_{\mathbf{a}}$. All three vectors live in the weighted projected-rollout space of Equation (2), and both errors are measured against the same y. The reverse triangle inequality gives

$$|e_{\tilde{h}} - e_h| = \big|\, \|v - y\|_2 - \|u - y\|_2 \,\big| \tag{13}$$

$$\leq \|(v - y) - (u - y)\|_2 = \|v - u\|_2 = \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{14}$$

Equality holds exactly when $u - y$ and $v - y$ are positively colinear. The adverse increase obeys the same bound because $(e_{\tilde{h}} - e_h)_+ \leq |e_{\tilde{h}} - e_h|$.

Proposition 3 (Fixed-pool selection certificates)**.** *In the setting of Proposition 2, if the clean winner w is unique and*

$$\min_{j \neq w} \big(C_j - C_w - b_j - b_w\big) > 0, \tag{15}$$

then the perturbed branch has the same winner. If $\mathcal{E}$ is the clean top-k elite set and

$$\min_{i \in \mathcal{E},\, j \notin \mathcal{E}} \big(C_j - C_i - b_j - b_i\big) > 0, \tag{16}$$

then the perturbed branch has exactly the same elite set.

Corollary 1 (Conditional CEM stability)**.** *Consider clean and perturbed CEM runs initialized by the same proposal and sampled with common random numbers. While their ordered candidate pools remain aligned, if Equation* (16) *holds at the current iteration, both runs fit the same next proposal. If it holds at every iteration, their pools and proposals remain aligned by induction; since the evaluated solver returns the final proposal mean as its action, the selected actions then coincide as well.*

Proof of Proposition 2. For one candidate, suppress the index and factor the squared-distance difference:

$$|\tilde{C} - C| = \left| \|\tilde{x} - g\|_2^2 - \|x - g\|_2^2 \right| \tag{17}$$
$$= \left| \langle \tilde{x} - x, \tilde{x} + x - 2g \rangle \right| \tag{18}$$
$$\leq \|\tilde{x} - x\|_2 \, \|\tilde{x} + x - 2g\|_2 \tag{19}$$
$$\leq r\big(\|x - g\|_2 + \|\tilde{x} - g\|_2\big) = b. \tag{20}$$

The first inequality is Cauchy–Schwarz and the second is the triangle inequality. Thus, for nominal winner w and competitor j,

$$\tilde{C}_j - \tilde{C}_w \geq C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \geq C_j - C_w - b_j - b_w.$$

For the perturbed winner $\tilde{w}$,

$$C_{\tilde{w}} \leq \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \leq \tilde{C}_w + b_{\tilde{w}} \leq C_w + b_w + b_{\tilde{w}},$$

which proves Equation (6); nonnegativity follows from the optimality of w on the nominal pool.

For Proposition 3, positivity of Equation (15) makes every perturbed competitor strictly more costly than w. The same argument for every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (16) preserves the exact elite set. Strict inequalities avoid relying on implementation-specific tie breaking.

Proof of Corollary 1. Index the branches by $b \in \{\mathrm{c}, \mathrm{p}\}$ (clean, perturbed) and iterations by t. Let ϕ_t^b collect branch b's proposal parameters (mean and variance), let ω_t be the shared random numbers, and let $\mathcal{A}_t^b = T(\phi_t^b, \omega_t)$ be the sampled candidate pool, where T is the deterministic sampling map. Branch b selects the elite set $\mathcal{E}_t^b$ as the k lowest-cost candidates under its own costs and updates $\phi_{t+1}^b = U(\{\mathbf{a}^j : j \in \mathcal{E}_t^b\})$, where U is deterministic and permutation invariant; the evaluated solver's unweighted elite mean and standard deviation satisfy this. The induction step is

$$\phi_t^{\mathrm{c}} = \phi_t^{\mathrm{p}} \implies \mathcal{A}_t^{\mathrm{c}} = \mathcal{A}_t^{\mathrm{p}} \overset{\text{Equation (16)}}{\implies} \mathcal{E}_t^{\mathrm{c}} = \mathcal{E}_t^{\mathrm{p}} \implies \phi_{t+1}^{\mathrm{c}} = \phi_{t+1}^{\mathrm{p}},$$

and the base case $\phi_0^{\mathrm{c}} = \phi_0^{\mathrm{p}}$ holds by assumption, which aligns pools and proposals for as long as the certificate holds at every completed iteration. The evaluated solver returns the final proposal mean as its action, so aligned proposals return identical actions. An implementation that instead returns the best final candidate additionally requires the top-1 condition Equation (15) at the last iteration. If a certificate fails, equality of the elite sets is unknown; the induction stops and no later shared-pool statement is made.

Relation to the weighted rollout radius. The planner bound uses the displacement at the cost horizon. For the weighted stack in Equation (3), $\sqrt{\alpha_H}\, r_j \leq \mathrm{ACPC}_H$ whenever $\alpha_H > 0$. The planner experiments record r_j directly, avoiding the looser $1/\sqrt{\alpha_H}$ conversion. This does not require a future target, but it does require evaluating the candidate under both matched histories.

Sharpness without additional assumptions. Neither principal bound admits a uniformly smaller right-hand side from the available quantities alone. For a unit vector u, common target $Y = 0$, and two stacked predictions au and bu with $a, b \geq 0$, the reverse-triangle bound in Equation (4) holds with equality. Likewise, setting $g = 0$, $x = au$, and $\tilde{x} = bu$ makes $|\tilde{C} - C| = |b - a|(a + b) = r(\|x - g\|_2 + \|\tilde{x} - g\|_2)$.

B Local-Geometry Case-Study Definitions

This section defines the descriptive quantities reported by the case study in Section 4.2. The case study uses 128 logged PushT histories; for each, one clean view and 18 perturbed views are generated—six draws at each probe standard deviation in $\{0.01, 0.04, 0.08\}$. The t-SNE display reproduces the deterministic per-panel projections of the original Appendix visualization.

Let $\mathcal{V} \in \{\mathrm{enc}, \mathrm{roll}\}$ denote either the encoder output or the representation after eight autoregressive rollout steps (the protocol horizon H=8; Section 3.1). For anchor history i, let $v_{i,\mathcal{V}}^{(0)}$ be its clean representation and $v_{i,\mathcal{V}}^{(m)}$, $m = 1, \ldots, M$, the representations of M perturbed views. In the rollout space, the clean and perturbed views of

each anchor use the same recorded action sequence. We define the sampled visual radius, nearest-clean spacing, and their ratio as

$$r_{i,\mathcal{V}}^{\mathrm{vis}} = \max_{1\le m\le M} \left\| v_{i,\mathcal{V}}^{(m)} - v_{i,\mathcal{V}}^{(0)} \right\|_2,$$
$$d_{i,\mathcal{V}}^{\mathrm{NN}} = \min_{j\neq i} \left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2, \qquad \rho_{i,\mathcal{V}}^{\mathrm{local}} = \frac{r_{i,\mathcal{V}}^{\mathrm{vis}}}{d_{i,\mathcal{V}}^{\mathrm{NN}}}. \tag{21}$$

The figures abbreviate $\rho_{i,\mathcal{V}}^{\mathrm{local}}$ as r/NN. When this ratio is below one, every sampled perturbed view of anchor i lies closer to its clean representation than that representation lies to the nearest other clean anchor. A ratio of at least one means that the largest sampled displacement reaches or exceeds this nearest-clean distance. This is a geometric comparison; the nearest clean anchor need not lie across a semantic or task-relevant boundary.

The ratio uses only anchor i's perturbation radius. It does not account for the perturbation radius around the neighboring anchor. We therefore add a symmetric test: the enclosing ball around anchor i is *fully disjoint* if it is separated from the enclosing ball around every other anchor:

$$\left\| v_{i,\mathcal{V}}^{(0)} - v_{j,\mathcal{V}}^{(0)} \right\|_2 > r_{i,\mathcal{V}}^{\mathrm{vis}} + r_{j,\mathcal{V}}^{\mathrm{vis}} \quad \text{for every } j \neq i. \tag{22}$$

Across the sampled anchors, we report three summaries: the median r/NN ratio; the fraction satisfying $r_{i,\mathcal{V}}^{\mathrm{vis}} < d_{i,\mathcal{V}}^{\mathrm{NN}}$; and the fraction satisfying the fully-disjoint condition in Equation (22). The second is a one-sided nearest-center test, whereas the third is a symmetric certificate for the enclosing balls in the original high-dimensional representation. It is unrelated to whether the qualitative t-SNE display ellipses overlap. Because these radii are estimated from sampled views, their values depend on the number of perturbation draws, anchor density, and representation scale. Encoder and final-step comparisons also omit intermediate predictions. Neither summary bounds prediction-error or planner-cost changes; those links are provided by pairwise ACPC in Section 3.1.

C Evaluation and Diagnostic Protocol

This appendix specifies the fixed evaluation and diagnostic protocol used in the main tables.

Evaluation grid. LeWM is evaluated on PushT, TwoRoom, Reacher, and Cube with evaluation seeds 42/43/44 and 100 episodes per seed. The main evaluation perturbs observation pixels with Gaussian noise at standard deviation 0.08 while keeping the goal image clean. Checkpoints trained with noise use full-sequence input-side Gaussian-noise augmentation with $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$.

PLDM protocol. PLDM uses the same four tasks, nine-checkpoint Gaussian-noise training grid, evaluation seeds, trajectory count, paired-history construction, $H = 8$ rollout statistic, and SR definition. For cross-task analysis, we divide raw IR by the value of the corresponding unaugmented PLDM checkpoint. We then select thresholds on the other three PLDM tasks. Success rates from the evaluation task never enter threshold selection. Thresholds are selected separately within each model family; we do not assume that the families share raw thresholds.

Success-rate criterion. Within each task and training run, let P_{base} and P_{best} be the success rates at evaluation noise $\sigma = 0.08$ for the unaugmented checkpoint and the best checkpoint in the sweep. A checkpoint meets the success-rate criterion when its noisy-observation success rate is at least $P_{\mathrm{base}} + 0.8(P_{\mathrm{best}} - P_{\mathrm{base}})$ and its clean success rate is no more than five percentage points below that of the unaugmented checkpoint. Statements about recovered levels in Figure 3 require this criterion to hold in a majority of runs.

IR protocol. We compute IR separately for each task, training run, and checkpoint. For anchor i, let $u_{i,0}^{\mathrm{obs}}$ be the embedding of the last clean history observation in the projected space selected by Π, and $u_{i,1:H}^{\mathrm{obs}}$ the embeddings of the next H actually observed clean frames in that space; the clean-motion scale is

$$s_i = Q_{0.50}\Big(\big\{ \|u_{i,k}^{\mathrm{obs}} - u_{i,k-1}^{\mathrm{obs}}\|_2 \big\}_{k=1}^{H} \Big). \tag{23}$$

Table 2: Implemented state-coordinate labels for SR. The state is taken at the eighth observed future step and standardized coordinate-wise with full-dataset statistics. Counts give eligible and skipped anchors in the fixed 100-anchor evaluation; these labels are not semantic or action-relevance annotations.

Task	Logged state field (dim.)	Implemented label $\ell(y)$	Eligible / skipped
TwoRoom	`pos_agent` (2)	One median bit from the standardized agent x coordinate (slice `pos_agent[0:1]`).	61 / 39
PushT	`state` (7)	Three median bits from standardized block x, block y, and block angle (slice `state[2:5]`).	98 / 2
Reacher	`observation` (6)	Three median bits from the two standardized joint-velocity coordinates and their Euclidean norm (slice `observation[4:6]`).	100 / 0
Cube	`observation` (28)	Three median bits from the first two coordinates and the Euclidean norm of $r = \bar{o}_{0:3} - \bar{o}_{25:28}$, where $\bar{o}$ is the standardized observation.	100 / 0

Each anchor contributes one weighted radius over the rollout horizon. Perturbation draws use Gaussian noise on observations at $\sigma = 0.08$, matching the behavioral evaluation severity; under blur or resize, draws apply that shift instead. The default uses $H = 8$ and uniform weights $\alpha_k = 1/H$. We divide each radius by the anchor's q50 clean observed-transition scale, including the transition from the final history observation to the first future observation. We then average multiple noise draws within each anchor and take q90 across anchors. This gives raw IR in Equation (9), which SR uses as its same-history reference. For sweep plots and cross-task threshold selection within each model family, we divide raw IR by the corresponding unaugmented value as in Equation (11) before computing thresholds or run summaries. The numerical stabilizers are $\varepsilon_{\text{norm}} = 10^{-8}$ in Equation (8) and $\varepsilon_{\text{score}} = 10^{-12}$ in Equation (12). Table 1 reports how the checkpoint-level IR reduction depends on the rollout horizon and the summary quantile.

Table 1: Horizon and quantile sensitivity of the IR reduction at fixed evaluation noise $\sigma = 0.08$. Each entry is the median, over the three training runs, of the ratio between the IR of the checkpoint trained at the highest augmentation level ($\sigma_{\max} = 0.08$) and that of the unaugmented checkpoint; lower means a larger reduction. These values are descriptive and do not retune any threshold.

	$q = 0.90$ by horizon				$H = 8$ by quantile		
Task	H1	H2	H4	H8	q80	q90	q95
TwoRoom	0.05	0.04	0.05	0.06	0.06	0.06	0.06
PushT	0.06	0.07	0.06	0.09	0.07	0.09	0.09
Reacher	0.02	0.03	0.03	0.02	0.02	0.02	0.02
Cube	0.04	0.03	0.04	0.04	0.03	0.04	0.04

SR protocol. SR follows Equation (10). Let $d_{0.35}$ be the q35 of all off-diagonal Euclidean distances among the standardized endpoint states; this radius, well below the median pairwise distance, keeps every selected pair genuinely nearby in state space while leaving most anchors an eligible neighbor with a different label (Table 2 reports the counts). Each eligible anchor chooses the nearest neighbor $j(i)$ with a different label and distance at most $d_{0.35}$. The predictor rolls both clean histories under the anchor actions $\mathbf{a}_i$; for the neighbor these are not the actions it actually executed, but reusing the anchor's actions keeps a single shared action sequence on both sides. The normalized distance between the two labeled histories is

$$D_i^{\text{diff}} = \frac{\left\| \bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)})) \right\|_2}{s_i + \varepsilon_{\text{norm}}}. \tag{24}$$

The pair passes only if this distance is strictly greater than raw q90 IR plus 0.10.

Table 2 makes the pairing rule explicit. The logged state is taken at the end of the observed eight-step future and standardized coordinate-wise by the dataset preprocessor. The neighborhood radius is the $d_{0.35}$ defined above, computed in that task's full standardized state vector. State-coordinate labels use median splits computed within the fixed 100-endpoint anchor batch (anchor seed 9101); each label therefore describes the endpoint reached by that trajectory's own recorded actions. After selecting a neighbor, we roll both starting histories under the anchor's actions for the latent-distance test. We skip an anchor only when no state with a different label lies inside the neighborhood. Pairing depends only on the logged states, so the eligible counts remain constant across checkpoints and training runs.

Table 3: Summary of the Gaussian-noise training sweep (nine checkpoints per task: no augmentation plus eight noise levels; Figure 3 shows every level). "Best" is the level with the highest mean planning success at evaluation noise $\sigma = 0.08$; arrows give the change from the unaugmented checkpoint to that level. Relative IR is lower-is-better, and SR is higher-is-better. The last column lists levels meeting the success-rate criterion (Section 4.1).

Task	No-aug. success (%)	Best success (%)	Best $\sigma_{\max}$	Relative IR	SR	Levels meeting criterion
TwoRoom	68.8	97.1	0.08	1.00→0.07	0.11→0.98	0.01–0.08
PushT	7.2	86.8	0.06	1.00→0.11	0.28→1.00	0.03–0.08
Reacher	18.2	83.3	0.07	1.00→0.03	0.37→0.99	0.03–0.08
Cube	43.1	66.0	0.03	1.00→0.26	0.07→0.98	0.03–0.07

Table 4: Relative reduction in out-of-group MAE from Base+ACPC$_8$ when predicting the error drift d on the unaugmented checkpoints (protocol and model definitions in Section 4.4). Base+Control$_8$ uses the per-cell oracle control; higher is better. The last column counts task–run cells in which Base+ACPC$_8$ is best.

Training run (seed)	vs. Base	vs. Base+Control$_8$	Task–run cells improved
3072	55.5%	51.4%	4/4
3073	60.8%	54.8%	4/4
3074	51.4%	47.7%	4/4
mean ± SD	55.9 ± 4.7%	51.3 ± 3.5%	12/12

Threshold grids. For cross-task evaluation, candidate thresholds are $t_{\mathrm{IR}} \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}$ for task-relative IR and $t_{\mathrm{SR}} \in \{0.80, 0.85, 0.90, 0.95\}$ for SR. Within each source-task subset, selection minimizes, in order, mean absolute recovery-onset mismatch, false-early count, and false-late count. It then maximizes balanced accuracy, prefers smaller t_{IR}, and finally prefers larger t_{SR}. The selected pair is applied unchanged to every held-out task. No held-out-task label or blur/resize outcome enters threshold selection.

Reproducibility. The scripts documented in `paper1/scripts/README.md` rebuild all summary figures and tables deterministically. Checkpoint-level diagnostics also require the released checkpoints and datasets. Machine-readable summaries record training seeds, evaluation seeds, and protocol hashes.

D Prediction-Error Scope and Controls

The common-future inequality in Proposition 1 has zero numerical violations in any logged eight-step or candidate-specific five-step task×run×training-condition cell; this is an implementation invariant, not an independent statistical success count.

Logged-future prediction and candidate planning are different estimands: the former predicts the error drift against an independent action-matched observed future, whereas the latter compares candidates from the actual planner pool. We do not infer one result from the other; Section 4.5 supplies the planner-facing evidence with same-pool, candidate-specific features.

The prediction-error analysis is restricted to the unaugmented checkpoints and the listed visual probes. Table 4 reports the per-run relative reductions, and Table 5 the underlying out-of-group MAE values with the selected controls. Every training run improves on all four tasks.

E Cross-Task Threshold Partitions

The 14 rows below are exhaustive: selecting one, two, or three of four tasks as sources creates $4 + 6 + 4$ directional partitions. Each evaluation task retains every training run and all nine checkpoints. The apparent sample size is therefore not inflated by treating checkpoint rows as independent observations.

The single-source Reacher exception in Section 4.6 is a tie-break in fit: under $(0.3, 0.95)$ Reacher's own onset mismatch is also at most one level, but calibrating on Reacher alone prefers the tighter $t_{\mathrm{IR}} = 0.1$ that fits it exactly and does not carry over. Recovered Reacher checkpoints reach relative IR well below 0.1, while the earliest recovered levels on the other tasks lie above it, so the tighter threshold rejects them and delays the accepted onset.

Table 5: Out-of-group MAE (16-fold leave-one-trajectory-group-out) for predicting the error drift d on the unaugmented checkpoints; lower is better, model definitions in Section 4.4. "Selected control" is the destroyed-action control chosen by the per-cell oracle; "paired wins" counts the folds (of 16) in which Base+ACPC$_8$ beats Base+Control$_8$.

Task	run (seed)	Base	Base +Control$_8$	Base +ACPC$_8$	selected control	paired wins
TwoRoom	3072	2.535	2.099	0.755	shuffled actions	15/16
TwoRoom	3073	2.336	2.015	0.866	shuffled actions	15/16
TwoRoom	3074	2.075	1.911	1.006	swapped actions	13/16
PushT	3072	2.068	2.068	1.762	shuffled actions	11/16
PushT	3073	1.969	2.008	1.315	zeroed actions	13/16
PushT	3074	1.909	1.833	1.439	zeroed actions	13/16
Reacher	3072	1.358	1.097	0.288	shuffled actions	16/16
Reacher	3073	1.399	0.793	0.247	shuffled actions	16/16
Reacher	3074	1.409	1.136	0.263	shuffled actions	16/16
Cube	3072	1.171	1.037	0.489	zeroed actions	14/16
Cube	3073	1.416	1.212	0.500	zeroed actions	14/16
Cube	3074	1.061	1.008	0.552	shuffled actions	14/16

F Cross-Stressor Pairs and Discordant Cases

Both discordant rows lie near the success-rate criterion: PushT run 3072 under resize and PushT run 3074 under blur each improve success rate by four percentage points, one point below the five-point criterion, while their IR–SR scores increase. The complete table shows these cases rather than only the aggregate classification metrics.

G Local Sensitivity to Gaussian Noise as a Mechanistic Interpretation

For a small isotropic observation perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$, first-order expansion of encoder E followed by rollout map G gives $\Delta G = J_G J_E \, \delta x + o(\|\delta x\|_2)$ with Jacobians evaluated at the clean input. Writing $A = J_G J_E$ and taking expectations of the leading term,

$$\mathbb{E}\big[\|A\,\delta x\|_2^2\big] = \mathrm{tr}\big(A\,\mathbb{E}[\delta x\,\delta x^\top]A^\top\big) = \sigma^2\,\mathrm{tr}(AA^\top) = \sigma^2\|A\|_F^2, \tag{25}$$

so $\mathbb{E}[\|\Delta G\|_2^2] \approx \sigma^2\|J_G J_E\|_F^2$ to first order as $\sigma \to 0$. Jacobian Frobenius norms have previously been used to quantify and regularize local representation sensitivity and contraction [57]. We estimate $\mathrm{tr}[(J_G J_E)^\top (J_G J_E)] = \|J_G J_E\|_F^2$ with Rademacher probes using the Hutchinson estimator [58]; JVPs avoid materializing the full Jacobian. This explains one mechanism by which training with Gaussian noise can contract ACPC: it reduces the composed local sensitivity of the encoder–predictor path (Figure 8). It is a local approximation, not a global robustness bound.

H Adaptive-CEM Selection-Regret Protocol

The selection-regret analysis uses no task-success labels. It evaluates the unaugmented and $\sigma_{\max} = 0.08$ checkpoints on 100 histories per task, training run, and training condition at perturbation severities $0.02, 0.05, 0.08$. Each CEM branch uses $K = 64$ candidates, eight elites, eight iterations, and a five-step model-prediction horizon. The clean and perturbed branches start from the same proposal and use common random numbers, but each branch updates its proposal from its own elites. The implemented cost is the mean-reduced squared error over embedding coordinates, i.e., $\|x - g\|_2^2/d$ for embedding dimension d; this common positive factor rescales every C_j and b_j identically and leaves the regret bound and both certificates unchanged.

For every candidate, we compute ACPC along that candidate's action sequence and summarize the pool by its q90 at horizons one and five. For clean plan $\mathbf{a}$ and perturbed-history plan $\tilde{\mathbf{a}}$, the prediction target is the clean-reference selection regret

$$\big[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})\big]_+.$$

Within each training run, ridge regressions are fitted on three tasks and evaluated on the fourth. MAE is computed on the log-transformed target and averaged equally across the four tasks excluded from fitting in turn.

Table 6: Joint IR/SR checkpoint screening with thresholds chosen on other tasks. Thresholds are selected on the threshold-selection tasks and applied unchanged to all test tasks. Metrics first average training runs within each test task and then weight tasks equally. Recovery-onset mismatch is measured in training-augmentation grid levels (one level is 0.01 in $\sigma_{\max}$).

Selection tasks	Test tasks	t_{IR}	t_{SR}	BA	P / R	Onset mismatch mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.3 / 1.0
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.6 / 1.0
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	2.9 / 5.0
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.6 / 1.0
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.3 / 1.0
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.3 / 1.0
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.3 / 1.0
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.7 / 1.0
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.7 / 1.0
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.7 / 1.0
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.3 / 1.0
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.3 / 1.0
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.3 / 1.0
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	1.0 / 1.0

Table 7: The same leave-one-task-out IR–SR screen in two model families. For each evaluation task, thresholds are selected on that model family's other three tasks and applied unchanged. Values average three independent training runs; chance is 0.5.

	Balanced accuracy	
Evaluation task	LeWM	PLDM
TwoRoom	0.935	0.875
PushT	0.917	0.571
Reacher	0.889	0.632
Cube	0.858	0.717

The reported relative MAE decrease is computed within each run before taking the mean and sample standard deviation across the three runs. This protocol evaluates prediction of a clean-model cost; it does not evaluate simulator return.

Table 8: IR–SR scores under blur and resize. For each task, thresholds are chosen on the other three Gaussian-noise tasks and then fixed; each pair compares the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart. "Discordant" counts pairs whose score-change sign disagrees with the predefined success criterion (Section 4.8).

Visual shift	Pairs	BA	Precision / recall	Spearman	Discordant
All pairs	24	0.889	0.882 / 1.000	0.835	2
Blur	12	0.875	0.889 / 1.000	0.873	1
Resize	12	0.900	0.875 / 1.000	0.746	1

Table 9: All 24 LeWM checkpoint pairs evaluated under blur and resize. $\mathrm{IR}_{\mathrm{rel}}$ and SR are measured for the checkpoint trained with Gaussian-noise augmentation ($\sigma_{\max} = 0.08$); ΔP and ΔS are its success-rate and IR–SR score changes relative to the unaugmented checkpoint. A pair is positive when $\Delta P \geq 5$ percentage points with at most a five-point clean loss. Daggers mark the two discordant pairs.

Task	Seed	Shift	$\mathrm{IR}_{\mathrm{rel}}$	SR	ΔP	ΔS	Outcome (success / IR–SR)
TwoRoom	3072	blur	0.499	0.885	55.7	1.670	positive / positive
TwoRoom	3072	resize	0.336	0.967	52.3	2.214	positive / positive
TwoRoom	3073	blur	0.781	0.262	36.0	0.729	positive / positive
TwoRoom	3073	resize	0.691	0.361	40.0	1.030	positive / positive
TwoRoom	3074	blur	0.636	0.541	37.7	1.212	positive / positive
TwoRoom	3074	resize	0.617	0.656	34.3	1.277	positive / positive
PushT	3072	blur	0.943	0.939	10.7	0.189	positive / positive
PushT	3072	resize	0.814	0.969	4.0	0.620	nonpositive / positive†
PushT	3073	blur	0.846	0.918	7.3	0.515	positive / positive
PushT	3073	resize	0.682	0.969	24.3	1.060	positive / positive
PushT	3074	blur	0.781	0.959	4.0	0.729	nonpositive / positive†
PushT	3074	resize	1.173	0.939	-19.7	-0.577	nonpositive / nonpositive
Reacher	3072	blur	0.155	0.990	50.7	2.375	positive / positive
Reacher	3072	resize	0.210	0.990	29.7	2.375	positive / positive
Reacher	3073	blur	0.176	0.990	52.3	2.375	positive / positive
Reacher	3073	resize	0.207	0.990	40.0	2.375	positive / positive
Reacher	3074	blur	0.190	0.990	44.7	2.375	positive / positive
Reacher	3074	resize	0.195	0.990	33.3	2.375	positive / positive
Cube	3072	blur	1.447	0.330	-2.7	-1.488	nonpositive / nonpositive
Cube	3072	resize	1.166	0.530	1.0	-0.552	nonpositive / nonpositive
Cube	3073	blur	1.577	0.260	2.3	-1.923	nonpositive / nonpositive
Cube	3073	resize	1.218	0.560	-1.3	-0.728	nonpositive / nonpositive
Cube	3074	blur	1.220	0.660	-3.0	-0.735	nonpositive / nonpositive
Cube	3074	resize	1.040	0.770	-2.3	-0.134	nonpositive / nonpositive

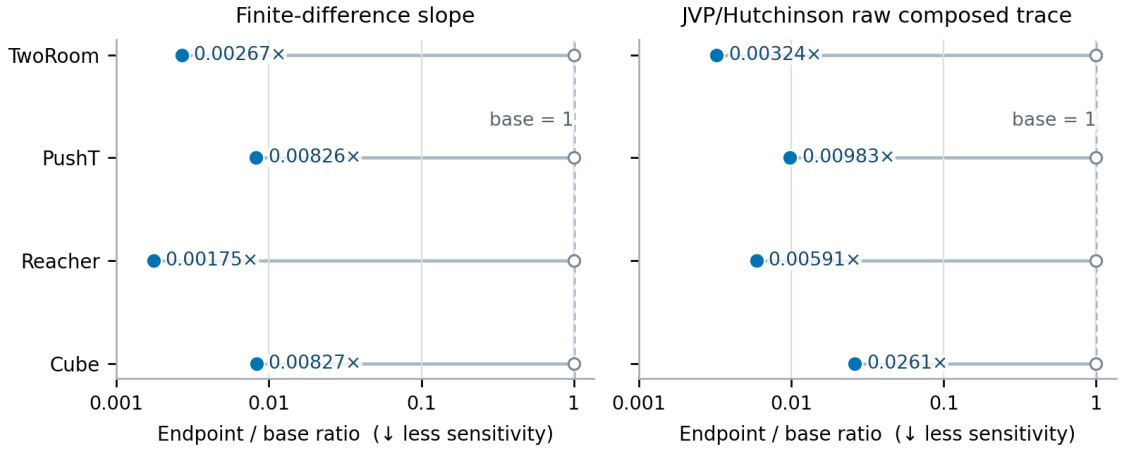

Figure 8: Local-sensitivity ratio between the Gaussian-noise-augmented and unaugmented checkpoints, estimated by finite differences and a JVP-based Hutchinson estimator of the composed Jacobian's squared Frobenius norm. For every task, both ratios are below one, indicating lower local sensitivity after augmentation but not establishing task performance.